

Ускорение инференса нейронных сетей: сравнительный анализ методов оптимизации

Итоговый отчёт

Авторы:

Фоменко Андрей Алексеевич
Акимов Тимур Арсенович
Старощук Богдан Павлович

Содержание

1	Введение	1
1.1	Постановка задачи	1
1.2	Исследуемые методы ускорения	1
2	Архитектура и инфраструктура	1
2.1	Модель: UNet	1
2.2	Функция потерь и метрики качества	2
3	Итоговые результаты GPU-бенчмарка	2
3.1	Конфигурация и протокол измерений	2
3.2	Итоговая таблица по методам	2
3.3	Относительные изменения относительно ONNX Runtime	3
3.4	Графики по финальному прогону V100	3
3.5	Выводы	3

1 Введение

Данный отчет описывает итоговые результаты проекта по ускорению инференса U-net. Ключевая задача это получение воспроизводимого бейслайна и применение техник оптимизации для дальнейшего анализа их эффективности.

1.1 Постановка задачи

В качестве целевой задачи выбрана бинарная сегментация изображений на датасете *Oxford-IIT*. Датасет содержит 7349 изображений кошек и собак с пиксельными масками. Мы бинаризуем маску на животное и фон.

Выбор этого датасета обусловлен небольшим размером (≈ 800 МБ), что сильно ускоряет эксперименты, наличием готовой загрузки через torchvision, достаточной сложностью для получения нетривиального бейслайна

1.2 Исследуемые методы ускорения

1. **Baseline FP16.** Запускаем из коробки на торче.
2. **torch.compile** Используем как наиболее простую оптимизацию без изменения архитектуры и весов, где ускорение достигается за счёт fusion операций, сокращения Python-overhead и более эффективного планирования kernel launch
3. **ONNX Runtime backend** Добавляется отдельный inference backend на базе ONNX Runtime, как альтернатива torch.compile. Модель экспортируется из PyTorch в ONNX-граф, после чего ORT применяет оптимизации графа, такие как удаление лишних узлов, constant folding и fusion операций, и исполняет граф через выбранный Execution Provider
4. **PTQ (Post-Training Quantization, int8)** Используем для быстрого сжатия модели без дообучения, где ускорение достигается за счёт int8-арифметики и уменьшения размера весов.
5. **QAT (Quantization-Aware Training, int8)** Дообучение может поднять качество квантизированной модели, то есть модель обучается с учётом будущей квантизации, что уменьшает quantization error
6. **Unstructured Pruning** Используем для удаления малозначимых весов и снижения эффективной сложности модели, тк после pruning и короткого finetune часто удаётся сохранить качество при меньшем числе активных параметров
7. **2:4 Semi-structured Sparsity** Используем как более аппаратно-ориентированный вариант pruning, выбрали шаблон 2:4 тк он лучше согласован с ускорением sparse-операций на современных GPU

2 Архитектура и инфраструктура

2.1 Модель: UNet

Как было сказано во введении, качестве модели используется классический UNet для бинарной сегментации. Архитектура состоит из симметричного энкодера и декодера со skip-connections:

- **Энкодер:** 4 блока ($\text{MaxPool2d} \rightarrow \text{Conv-BN-ReLU} \times 2$), количество каналов: $64 \rightarrow 128 \rightarrow 256 \rightarrow 512$;

- **Боттлнек:** 512 каналов;
- **Декодер:** 4 блока (Upsample $\rightarrow$ concat $\rightarrow$ Conv-BN-ReLU $\times$ 2);
- **Выход:** 1×1 свёртка $\rightarrow$ логит бинарной маски.

Суммарное количество обучаемых параметров: **13,391,361**.

2.2 Функция потерь и метрики качества

Для обучения используется комбинированная функция:

$$\mathcal{L} = 0.5 \cdot \mathcal{L}_{\text{Dice}} + 0.5 \cdot \mathcal{L}_{\text{BCE}} \quad (1)$$

где

$$\mathcal{L}_{\text{Dice}} = 1 - \frac{2|\hat{y} \cap y| + \varepsilon}{|\hat{y}| + |y| + \varepsilon}, \quad \varepsilon = 10^{-6} \quad (2)$$

Метрики оценки качества:

- **Dice coefficient** (основная метрика)
- **IoU**
- **Pixel Accuracy**

Метрики скорости:

- latency P50/P95 при batch size = 1 (мс)
- throughput (изображений/с) при batch size = 16
- размер модели (МБ)

3 Итоговые результаты GPU-бенчмарка

3.1 Конфигурация и протокол измерений

Ключевые параметры эксперимента:

- V100 GPU
- число измерений latency -100 запусков после warmup
- throughput измерялся при batch size = 16
- метрики качества: Dice, IoU, Pixel Accuracy на одной и той же тестовой выборке

3.2 Итоговая таблица по методам

Таблица 1: Результаты экспериментов на V100

Метод	Precision	P50 (мс)	P95 (мс)	Throughput	Dice	Размер (МБ)
Baseline FP16	fp16	4.50	4.77	488.34	0.89435	26.78
torch.compile	fp16	3.00	3.21	735.98	0.89435	26.78
ONNX Runtime	fp16	3.10	3.24	587.16	0.89434	26.81
PTQ (int8)	int8	6.17	6.27	236.85	0.89395	13.53
Pruning (unstructured)	fp16	4.73	5.14	487.41	0.89730	26.78
2:4 Semi-structured	fp16	4.76	5.21	488.63	0.88517	26.78

3.3 Относительные изменения относительно ONNX Runtime

Для удобства интерпретации приведём относительные эффекты по скорости, размеру и качеству. В качестве точки отсчёта используется ONNX Runtime fp16:

Таблица 2: Относительные изменения относительно ONNX Runtime

Метод	Ускорение P50	Рост throughput	Сжатие размера	Δ Dice
Baseline FP16	0.69×	0.83×	1.00×	+0.0000
torch.compile	1.03×	1.25×	1.00×	+0.0000
PTQ (int8)	0.50×	0.40×	1.98×	−0.0004
Pruning (unstructured)	0.66×	0.83×	1.00×	+0.0030
2:4 Semi-structured	0.65×	0.83×	1.00×	−0.0092

3.4 Графики по финальному прогону V100

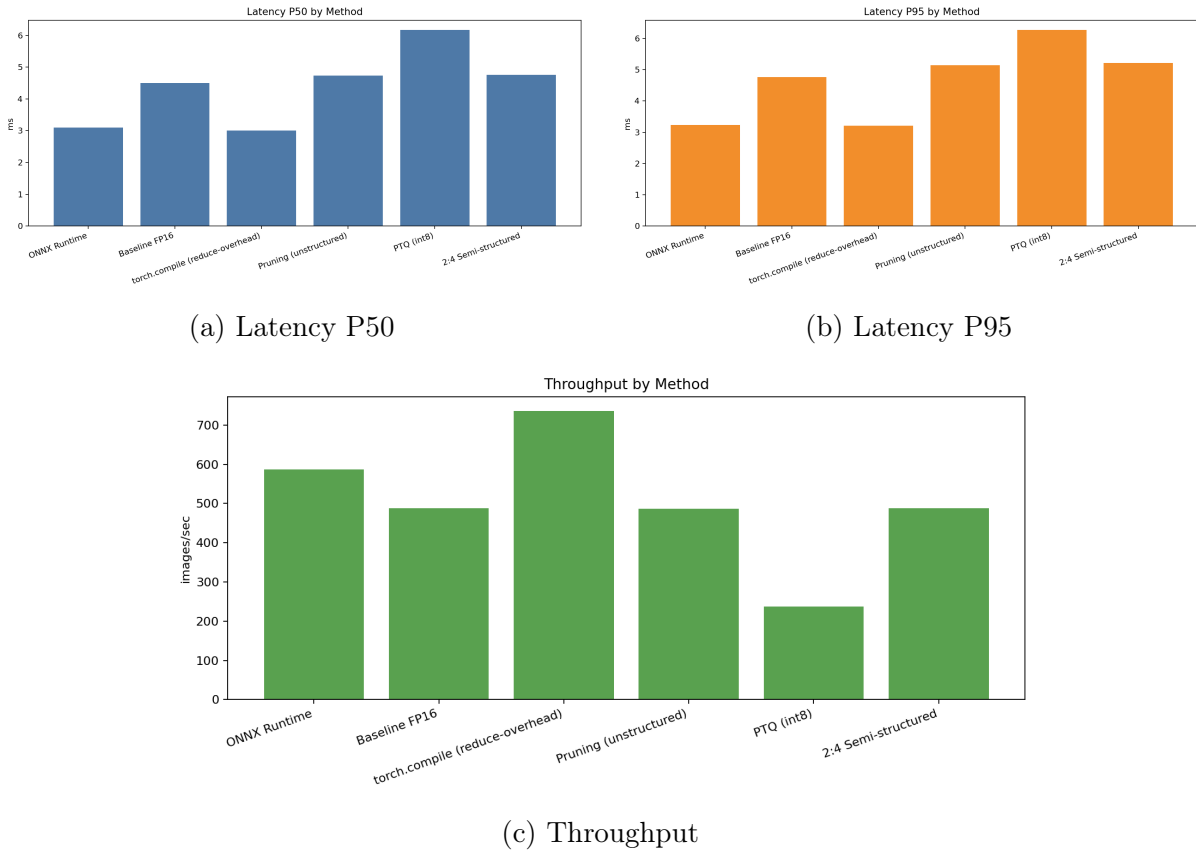


Рис. 1: Визуализация финальных метрик на V100

3.5 Выводы

Baseline FP16 Исходная FP16-модель демонстрирует P50 latency = 4.50 мс, P95 latency = 4.77 мс и throughput = 488.34 изображений/с при Dice = 0.89435. Относительно ONNX Runtime FP16 baseline заметно медленнее (0.69× по P50 и 0.83× по throughput), при этом качество фактически совпадает (Δ Dice $\approx$ 0).

`torch.compile torch.compile` показывает лучший скоростной результат в текущем наборе: P50 latency = 3.00 мс и P95 = 3.21 мс. Относительно ONNX Runtime это даёт ускорение $1.03\times$ по P50 и $1.25\times$ по throughput (735.98 против 587.16 изображений/с). Размер модели не меняется (26.78 МБ), а качество полностью сохраняется (Dice = 0.89435, то есть на уровне ONNX).

ONNX Runtime (fp16) ONNX Runtime FP16 используется как референсный backend для расчёта относительных изменений. Метод показывает P50 latency = 3.10 мс, P95 latency = 3.24 мс и throughput = 587.16 изображений/с при Dice = 0.89434. Это сильный и стабильный runtime baseline, от которого удобно сравнивать как квантизацию, так и методы разреживания.

PTQ (int8). После post-training quantization размер модели уменьшается с 26.81 МБ до 13.53 МБ, что даёт сжатие $1.98\times$ относительно ONNX FP16. По качеству PTQ почти не деградирует (Dice = 0.89395, Δ Dice = -0.0004), но по скорости в этой конфигурации метод проигрывает: P50 latency растёт до 6.17 мс ($0.50\times$ к ONNX), throughput падает до 236.85 изображений/с ($0.40\times$). То есть PTQ здесь выгоден в первую очередь как метод сжатия модели, а не как способ ускорения.

Pruning (unstructured). Unstructured pruning показывает P50 latency = 4.73 мс и throughput = 487.41 изображений/с, что ниже ONNX Runtime ($0.66\times$ по P50 и $0.83\times$ по throughput). Размер модели в dense чекпоинте не меняется (26.78 МБ), так как занулённые веса сохраняются в плотном формате. При этом по качеству это лучший результат в таблице: Dice = 0.89730 (Δ Dice = $+0.0030$ к ONNX).

2:4 Semi-structured Sparsity. 2:4 semi-structured sparsity в текущем прогоне не даёт выигрыша по скорости: P50 latency = 4.76 мс и throughput = 488.63 изображений/с ($0.65\times$ и $0.83\times$ относительно ONNX). Размер dense checkpoint также остаётся 26.78 МБ. По качеству метод заметно уступает остальным FP16-вариантам: Dice = 0.88517, что соответствует Δ Dice = -0.0092 к ONNX.

В итоге По итогам финального прогона на V100 лучший вариант по скорости – `torch.compile`: это единственный метод, который одновременно быстрее ONNX по latency и даёт заметный прирост throughput при том же качестве. ONNX Runtime остаётся сильным опорным backend с предсказуемым поведением и близкими метриками качества. PTQ уверенно решает задачу сжатия ($1.98\times$), но на данном стеке не ускоряет инференс. Методы pruning и 2:4 в текущей реализации также не дали runtime-выигрыша относительно ONNX; при этом unstructured pruning показал лучшую Dice-метрику, а 2:4 – наиболее заметную деградацию качества.